



SUMMER TRAINING REPORT

LangChain & LangGraph

Study Mate AI

Agentic study assistant using retrieval, tools, memory, and graph-based orchestration

Training Track	LangChain & LangGraph
Specialization	Data Science & AI
Student	Ziad Saied Ahmed
Student ID	20230113
Instructor name	Eng/ Rahma

Final Project: Study Mate AI

01

Table of Contents

Section	Title	Page
1.	Student Information & Report Scope	3
2.	Training Concepts	4–8
3.	Final Project – Study Mate AI	9
4.	Project Architecture	10
5.	LangGraph Workflow	11
6.	RAG Implementation	12
7.	Memory, Tools, Models & Persistence	13
8.	Application Demonstration & LangSmith	14
9.	Conclusion	15

Report Basis

This report consolidates the uploaded training sections, the StudyMate AI project archive, the supplied compiled graph, and the supplied application screenshot. Project-specific claims are limited to behaviour and technologies evidenced by those materials.

Evidence Boundary

No performance benchmark, accuracy percentage, or user-study result is reported because the supplied materials do not provide such measurements. The current Chroma database supplied with the project contains 56 stored embeddings across two indexed PDF sources: 28 for Data Analysis Basics.pdf and 28 for uploaded_material\Data Interviews.pdf.

02

Student Information & Report Scope

Field	Details
Student Name	Ziad Saied Ahmed
Student ID	20230113
Training Track	LangChain & LangGraph
Specialization	Data Science & AI
Final Project	Study Mate AI

Report Scope

The training covered the construction blocks of modern LLM applications: prompts and runnables, memory, tools, embeddings, RAG, agentic RAG, LangGraph orchestration, asynchronous execution, model routing through LiteLLM, and persistent memory with SQLAlchemy. The final project combines a subset of these concepts into a working local study assistant.

A key distinction is maintained throughout this report: a topic may have been studied during training without being part of the final implementation. For example, Prompt Templates, batch/stream execution, the LangGraph tools_condition helper, and Human-in-the-Loop were covered in training materials, while the project uses its own routing functions and does not contain a Human-in-the-Loop node.

03

Training Concepts — LangChain Fundamentals

LangChain Fundamentals

The introductory training material presents LangChain as an open-source framework that connects large language models with data and practical tasks. Its modular approach covers prompts, memory, retrieval, agents, and chains, reducing repetitive implementation work.

LLMs, Prompts and Output Control

The training introduced LLM types, token and context-window considerations, system prompts, prompt engineering, and output control. Prompt Templates were presented as a way to replace hard-coded instructions with reusable, role-based structures. Structured output and output parsers were also covered, including Pydantic and JSON-oriented approaches.

Runnables and Execution Methods

LangChain Runnables provide a common execution interface. The training materials explicitly distinguish invoke for a single request, batch for multiple inputs, and stream for incremental output. The project itself uses asynchronous ainvoke for graph execution and model/tool calls; it does not implement a batch or streaming response path in the supplied application code.

Concept	Training focus	Project status
Prompt Templates	Reusable dynamic prompts and role-based structure	Studied; project uses a centralized system-prompt constant
Structured Output	Schema-controlled model responses	Implemented through final-answer tool schema
invoke / batch / stream	Single, multi-input and incremental execution	Async ainvoke used; no batch/stream endpoint shown
LCEL / Runnables	Composable execution model	LangChain runnables underpin model/tool interfaces

04

Training Concepts — Memory, Tools & Agents

Memory

The training explains that an LLM does not automatically retain conversation history. It introduces RunnableWithMessageHistory, in-memory message history, session IDs, window memory, summary memory, hybrid memory, and a permanent-memory exercise using SQLite.

Tools and Agent Behaviour

Tool calling was taught as a mechanism that lets a model decide when external actions are needed. The ReAct agent concept combines reasoning with action in an iterative loop. Tool descriptions and docstrings provide the model with information about when and how a tool should be used. The training also included a SerpAPI-based search task.

Final Project Mapping

Training concept	Evidence in StudyMate AI
Short-term memory	LangGraph checkpoints preserve per-thread state; messages are accumulated with the graph state reducer.
Session / thread ID	The service passes thread_id for checkpoint history and user_id separately for durable memory.
Tools	Web search, term definition, internal-document retrieval, memory tools, and a schema-backed final-answer tool are registered.
Agent loop	The agent may call research/memory tools, then returns to the agent node for synthesis before finalization.
Exact-question cache	check_cache reuses an earlier exact answer within the same thread when available.

Important Implementation Distinction

The project does not simply attach a generic memory object to a chain. It combines LangGraph checkpointing for conversation state with a separate SQLAlchemy-backed store for durable student facts and thread metadata.

05

Training Concepts — Retrieval-Augmented Generation

RAG Fundamentals

The training presents RAG as a method for connecting an LLM to external knowledge so that responses can be grounded in retrieved evidence rather than relying only on static model knowledge. The core pipeline is described as chunking, indexing, retrieval, and augmentation/generation.

Document Loading and Chunking

PDF loaders convert source content into standardized Document objects with metadata such as source and page. Chunking reduces context-window pressure while preserving useful semantic continuity. The training also discusses semantic chunking and different splitter strategies.

Embeddings and Vector Databases

Embeddings convert text into numerical vectors in a semantic space. The training covers cosine similarity, vector databases, HNSW indexing, record management, MMR retrieval, threshold retrieval, Wikipedia retrieval, and multi-query retrieval.

Training versus Implementation

The training covers several retrieval strategies, but the supplied project implements a specific path: PyMuPDFLoader → RecursiveCharacterTextSplitter → BAAI/bge-m3 embeddings → persistent Chroma → MMR retrieval. The project configures k=2, fetch_k=10, lambda_mult=0.5 and cosine space.

06

Training Concepts — Agentic RAG

From Fixed Retrieval to Reasoning-Driven Retrieval

The Agentic RAG section contrasts traditional RAG, which follows a fixed pipeline, with an agentic approach that reasons about whether retrieval is necessary and whether another tool or direct answer is more appropriate. This makes retrieval one possible action inside a broader decision process.

Structured Output as a Final Action

The training specifically warns against forcing structured output on the first model call when the agent still needs to search or reason. Instead, a structured-output tool can be called as the final step. StudyMate AI follows this design: the model sees the schema-backed submit_final_answer tool, while the graph finalization code parses its JSON result into the API response.

Rate Limiting and Retries

The training also addresses provider RPM limits, synchronized access, expiration of old requests, and retry behaviour. In the supplied project, provider RPM values and LiteLLM retry/cooldown settings are declared in YAML. The application therefore separates model routing policy from application code.

What Is Actually Implemented

- The agent can choose internal-document retrieval, web search, term definition, or memory tools based on the query and system instructions.
- Tool results are appended to the graph state and returned to the agent node for synthesis.
- A bounded tool-round counter prevents indefinite tool loops; after the configured limit, the project falls back to a tool-free final-answer request.
- The final answer is normalized into answer text, source labels, and a confidence score through the final-answer schema.

07

Training Concepts — LangGraph, Async, LiteLLM, SQLAlchemy & HITL

LangGraph

The training describes LangGraph as a graph-based orchestration framework for agentic applications. Its core concepts include shared state, nodes, edges, conditional routing, reducers, checkpoints, and thread IDs. ToolNode executes tools selected by a model, while tools_condition is presented as a pre-built routing helper.

Asynchronous Execution

The asynchronous section explains async and await for non-blocking I/O, along with asynchronous invocation and streaming patterns. The project uses async graph execution, async tools, async database-backed checkpointing, and asyncio.to_thread around synchronous database/search work.

LiteLLM and YAML

The training introduces LiteLLM as a compatibility and routing layer and demonstrates YAML-based model configuration. The project runs a local LiteLLM proxy at port 4000, exposes model aliases to LangChain, enables local caching, uses usage-based routing, and defines retry/cooldown behaviour in config.yaml.

SQLAlchemy and Human-in-the-Loop

SQLAlchemy/ORM concepts were studied and are used in the final project for users, threads, and long-term memories. Human-in-the-Loop was also studied through interrupt/resume patterns, but no interrupt/resume node or HITL approval flow appears in the supplied StudyMate AI implementation; it is therefore treated as a training concept only.

08

Final Project — Study Mate AI

Project Goal

Study Mate AI is a local browser-based study assistant designed to answer student questions using a combination of conversation state, local course documents, external web search, and persistent student memory. The project README identifies it as an agentic RAG study assistant built with LangGraph, LiteLLM, Chroma, and SQLAlchemy long-term memory.

Main Technologies

Layer	Technology / component	Role in project
UI	HTML / CSS / JavaScript	Browser chat, thread sidebar, PDF upload, learner selector
API	FastAPI + Uvicorn	HTTP endpoints for users, threads, chat, and PDF upload
Orchestrati on	LangGraph	Stateful workflow, conditional routing, tool execution, checkpointing
Models	LiteLLM + ChatOpenAI clients	Routes agent and summarization model aliases through a local proxy
RAG	PyMuPDF + RecursiveCharact erTextSplitter + BAAI/bge-m3 + Chroma	Ingests PDFs and retrieves relevant chunks
Web	SerpAPI + httpx Wikipedia API	Current web search and concise term definition
Persistence	SQLite + SQLAlchemy + AsyncSqliteSaver	Durable user data, threads, long-term memory, and graph checkpoints

Why LangGraph Was Selected

The project requires more than a linear prompt chain: it checks a cache, lets an agent decide whether to use tools, loops tool results back into the agent, finalizes a structured answer, and optionally summarizes long checkpoints. LangGraph provides the state and conditional-routing model used to express these transitions.

09

Project Architecture

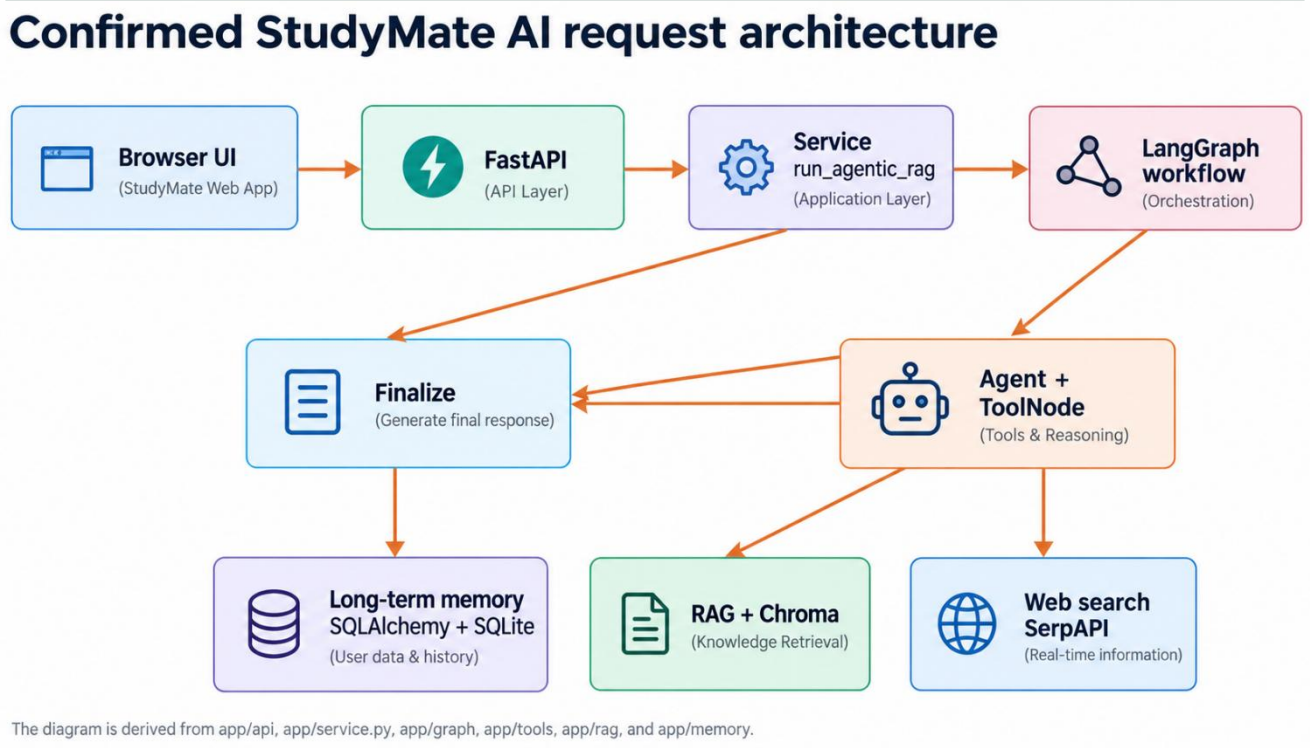


Figure 1. StudyMate AI request architecture reconstructed from the supplied project implementation.

Request Flow

- The browser sends a message, thread_id, and user_id to POST /api/chat.
- FastAPI validates the selected user and checks thread ownership before calling run_agentic_rag.
- The service establishes the user/thread records and invokes the compiled LangGraph with thread_id and user_id in the configurable runtime context.
- The graph begins at check_cache. A cache hit can finish directly; otherwise the agent node prepares conversation summary and relevant long-term memory context before calling the tool-enabled model.
- If the model requests a research or memory tool, ToolNode executes it and routes the result back to the agent. The finalize node then extracts the user-facing answer and metadata.
- When the retained message set becomes longer than the configured threshold, summarize_memory compresses older history and keeps a bounded recent window.

The supplied graph and source code contain a node named finalize, not final review. This report therefore uses the implementation name rather than introducing a new architecture element.

10

LangGraph Workflow

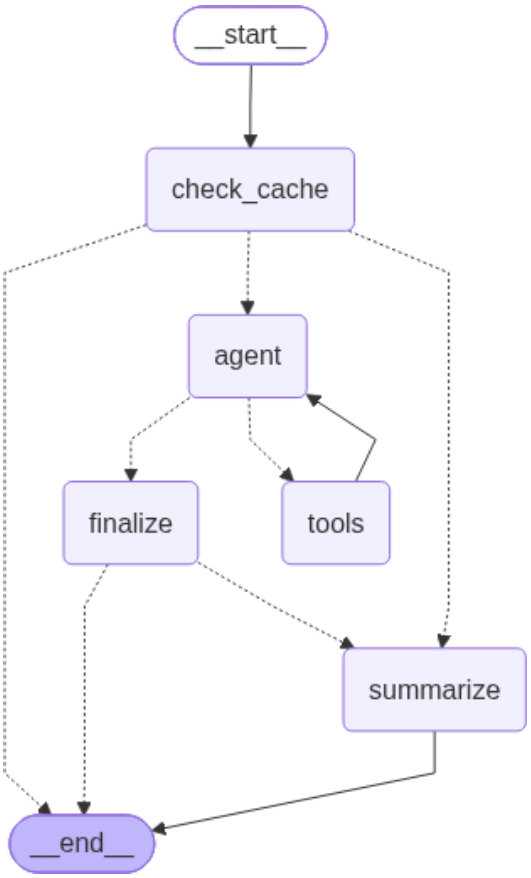


Figure 2. Supplied compiled LangGraph workflow for StudyMate AI.

Node / transition	Confirmed function
START → check_cache	Every incoming message first reaches the exact-question cache check.
check_cache → agent / summarize / END	A cache miss goes to the agent; a hit can finish or trigger summary compression when the message threshold is exceeded.
agent → tools / finalize	The model either requests registered tools or is routed to finalization.
tools → agent	ToolNode executes the selected research/memory tool and returns the result to the agent for synthesis.
finalize → summarize / END	The answer is extracted; long checkpoints may be summarized before termination.

11

RAG Implementation

Knowledge Sources

The project includes Data Analysis Basics.pdf as the default knowledge base and retains uploaded PDFs under uploaded_material/. The supplied Chroma database confirms two indexed sources at the time of packaging: Data Analysis Basics.pdf and uploaded_material\Data Interviews.pdf. The separate Sec RAG.pdf file is present in the archive, but it is not the default startup knowledge base and is not counted among the two sources confirmed in the persisted Chroma database.

Ingestion Pipeline

- PyMuPDFLoader extracts PDF pages into LangChain Document objects.
- RecursiveCharacterTextSplitter uses chunk_size=1000 and chunk_overlap=150.
- HuggingFaceEmbeddings loads BAAI/bge-m3 with normalized embeddings.
- Chroma persists the collection locally under studymate_chroma_db using cosine space and HNSW indexing.
- SQLRecordManager tracks indexed source chunks and supports incremental indexing.

Retrieval Path

The internal-document tool calls the global retriever asynchronously. The retriever uses Maximal Marginal Relevance (MMR) with k=2, fetch_k=10, and lambda_mult=0.5. Retrieved chunks are formatted with source and page metadata before being returned to the agent. This makes the RAG component a tool inside the agentic workflow rather than a mandatory first step for every question.

RAG Implementation Summary

Stage	Implementation
Documents	PDF pages from bundled/uploaded course material
Chunks	RecursiveCharacterTextSplitter, 1000 / 150
Embeddings	BAAI/bge-m3, normalized
Vector DB	Chroma, persistent, cosine/HNSW
Retrieval	MMR, k=2, fetch_k=10, lambda=0.5
Generation	Agent model uses retrieved evidence before finalization

12

Memory, Tools, Models & Persistence

Two Complementary Memory Layers

StudyMate AI separates short-term conversational state from durable student information. LangGraph checkpointing stores the state of a thread in studymate_checkpoints.sqlite.
Separately, SQLAlchemy models persist users, user_threads, and long_term_memories in studymate_long_term_memory.db. The project also maintains a sidebar history through user-thread records.

Tools

Tool	Purpose
retrieve_internal_documents	Semantic retrieval from the local Chroma knowledge base
web_search	Google search through SerpAPI; returns the first two organic results
define_term	Concise term lookup through the Wikipedia REST summary endpoint
remember_user_preference	Stores an explicit durable student fact
recall_user_preference / recall_all_preferences	Retrieves one or all durable student facts
submit_final_answer	Schema-backed final answer with answer, sources_used, and confidence_score

Multi-Model Architecture

The supplied config.yaml defines an agent-pool containing Groq gpt-oss-120b and OpenRouter liquid/lfm-2.5-2.6b:free. A separate summary-pool uses Gemini gemini-3.5-flash-lite. The LangChain application connects to the local LiteLLM proxy rather than embedding provider-specific model endpoints in the application logic.

Persistence and User Isolation

The browser supports multiple local learner profiles. Thread ownership is checked before a conversation is returned or continued, and durable memories are keyed by user ID. This creates separation between a student’s chat threads and long-term facts while sharing the indexed course-material collection.

13

Application Demonstration & LangSmith

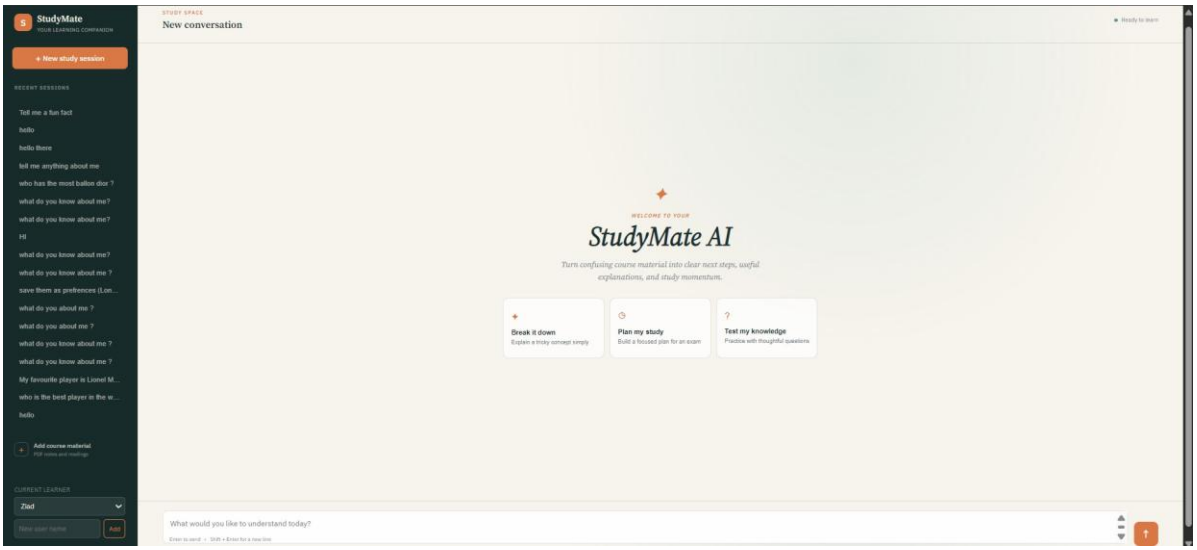


Figure 3. Supplied StudyMate AI application screenshot showing the browser interface.

What the Screenshot Demonstrates

- A left sidebar for recent study sessions and a New study session action.
- A learner selector and new-user form, consistent with the multi-profile implementation in the project source.
- A course-material upload action for PDF files.
- A central StudyMate AI welcome area with suggested study prompts.
- A chat composer for submitting student questions.

LangSmith Tracing

The project enables LangSmith tracing during application startup and sets the project name to Study Mate AI. The supplied implementation therefore establishes LangSmith as the tracing/observability layer for LangChain activity. The training material describes tracing as tracking requests and interactions within a conversation. Because no LangSmith dashboard export or trace screenshot was supplied, this report does not claim numerical execution-time, token-usage, or performance measurements.

Operational Observations from the Code

The application also writes structured logs, starts the LiteLLM proxy during startup, initializes persistent memory, indexes the default knowledge base, and saves the compiled graph visualization. These are implementation facts rather than measured performance outcomes.

14

Conclusion

The summer training developed a practical understanding of the components required to build modern LLM applications, progressing from LangChain fundamentals and prompt design to memory, tools, embeddings, RAG, agentic RAG, and graph-based orchestration. The training also introduced asynchronous programming, LiteLLM model routing, SQLAlchemy persistence, and Human-in-the-Loop patterns.

The Study Mate AI project applies these ideas as a coherent system. A FastAPI browser application sends requests to a LangGraph workflow that can reuse cached answers, reason about tool use, retrieve relevant course material, search the web, recall or store durable student facts, and finalize the response through a structured schema. LangGraph checkpoints preserve thread-level conversation state, while SQLAlchemy and SQLite provide durable user and memory records. LiteLLM provides a model-routing layer with separate agent and summarization pools, and Chroma supports persistent local RAG retrieval.

The main technical outcome is the integration of these components into one stateful agentic workflow rather than a collection of independent demonstrations. The supplied project materials show how retrieval, external tools, memory, persistence, and model routing can cooperate within a controlled graph while keeping the implementation modular and traceable.

Final Project: Study Mate AI

Training Track: LangChain & LangGraph

Specialization: Data Science & AI

Student: Ziad Saied Ahmed — 20230113